Thomas Algorithm

The **Thomas Algorithm**, also known as the **Tridiagonal Matrix Algorithm (TDMA)**, is a specialized direct solver for linear systems of the form $A\mathbf{x} = \mathbf{b}$, where A is a tridiagonal matrix. It is essentially a stripped-down version of Gaussian Elimination that exploits the banded structure of the coefficient matrix, reducing both memory and arithmetic cost dramatically.

In general Gaussian Elimination, the algorithm operates on a full $N \times N$ matrix, requiring $\mathcal{O}(N^3)$ operations and $\mathcal{O}(N^2)$ storage. For a tridiagonal system, however, only three diagonals carry non-zero entries. The Thomas Algorithm discards the full-matrix machinery entirely and works exclusively with three one-dimensional vectors:

- `lower` — the sub-diagonal (lower diagonal) coefficients,
- `main` — the main diagonal coefficients,
- `upper` — the super-diagonal (upper diagonal) coefficients.

This representation reduces storage from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ and operation count from $\mathcal{O}(N^3)$ to $\mathcal{O}(N)$, making it the standard choice for the implicit finite-difference schemes that produce tridiagonal systems at every time step — such as the Laasonen and Crank-Nicolson methods applied throughout this project.

The algorithm proceeds in two passes: a **Forward Elimination** phase that sweeps downward to eliminate the sub-diagonal entries, and a **Back Substitution** phase that recovers the solution vector from the resulting upper-bidiagonal system.

Forward Elimination: Matrix Transformation

The general tridiagonal system $Ax = b$ of size N takes the following form before elimination:

$$\begin{bmatrix} d_0 & u_0 & & & \\ \ell_1 & d_1 & u_1 & & \\ & \ell_2 & d_2 & \ddots & \\ & & \ddots & \ddots & u_{N-2} \\ & & & \ell_{N-1} & d_{N-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{N-1} \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ \vdots \\ b_{N-1} \end{bmatrix} \tag{1}$$

At each step $i = 1, 2, \ldots, N-1$, the elimination factor is:

$$f_i = \frac{\ell_i}{d_{i-1}} \tag{2}$$

and the main diagonal and right-hand side are updated as:

$$d_i^* = d_i - f_i \cdot u_{i-1}, \qquad b_i^* = b_i - f_i \cdot b_{i-1} \tag{3}$$

After Forward Elimination completes, the lower diagonal is eliminated and the system reduces to an upper bidiagonal form:

$$\begin{bmatrix} d_0 & u_0 & & & \\ & d_1^* & u_1 & & \\ & & d_2^* & \ddots & \\ & & & \ddots & u_{N-2} \\ & & & & d_{N-1}^* \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{N-1} \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1^* \\ b_2^* \\ \vdots \\ b_{N-1}^* \end{bmatrix} \tag{4}$$

Note that d_0 and b_0 serve as the anchor of the sweep and remain unchanged, while u_i is unmodified throughout. The resulting upper bidiagonal system is immediately ready for Back Substitution.

C++ Implementation Of `Forward Elimination`

The `ForwardElimination` function implements the forward sweep of the **Thomas Algorithm**. Starting from row $i = 1$ and iterating to $i = N - 1$, it computes the elimination factor $f_i = \ell_i / d_{i-1}$, updates the diagonal as $d_i \leftarrow d_i - f_i \cdot u_{i-1}$, modifies the RHS as $b_i \leftarrow b_i - f_i \cdot b_{i-1}$, and zeros out the sub-diagonal entry ℓ_i. The result is an upper bidiagonal system ready for back substitution.

```
void ThomasAlgorithm::ForwardElimination(
        TridiagonalMatrix& TridiagonalMatrix , RHS& RHS_Obj)
{
    const std::size_t NumRows = TridiagonalMatrix.SizeM();

    if (NumRows == 0)
    {
        throw std::invalid_argument(
            "ForwardElimination: diagonal vectors must not be empty.");
    }

    if (TridiagonalMatrix.SizeL() != NumRows
        || TridiagonalMatrix.SizeU() != NumRows
        || RHS_Obj.GetSize() != NumRows)
    {
        throw std::invalid_argument(
            "ForwardElimination: all diagonal vectors and RHS must have the same size.")
    }


for (std::size_t RowIndex = 1; RowIndex < NumRows; ++RowIndex)
    {
    const double OriginalDiagonalValue = TridiagonalMatrix.GetMidValue(RowIndex - 1);

    if (std::fabs(OriginalDiagonalValue) < 1.0e-14)
    {
        throw std::runtime_error(
```

```
            "ForwardElimination: zero or nearly zero diagonal entry detected.");
    }

    const double Factor =TridiagonalMatrix.GetLValue(RowIndex)/ OriginalDiagonalValue;

    const double NewDiagonal =TridiagonalMatrix.GetMidValue(RowIndex)- Factor * Tridiago

    TridiagonalMatrix.SetMValue(RowIndex,NewDiagonal);

    const double NewRHS =RHS_Obj.GetValue(RowIndex)- Factor * RHS_Obj.GetValue(RowIndex

    RHS_Obj.SetValue(RowIndex,NewRHS);

    TridiagonalMatrix.SetLValue(RowIndex,0.0);

    }

}
```

C++ Implementation Of `Back Substitution`

Once forward elimination reduces the system to upper-bidiagonal form, back substitution recovers the solution vector. The last unknown is solved directly:

$$x_{N-1} = \frac{b^*_{N-1}}{d^*_{N-1}}$$

Then, for $i = N-2$ down to 0:

$$x_i = \frac{b^*_i - u_i \cdot x_{i+1}}{d^*_i}$$

The loop variable uses `std::ptrdiff_t` instead of `std::size_t` to allow an explicit `>= 0` termination condition and avoid integer underflow on the unsigned decrement. A zero-pivot check is performed at each step before the division.

```
void ThomasAlgorithm::BackSubstitution(
    const TridiagonalMatrix& TridiagonalMatrix,
    const RHS& RHS_Obj,
    Field1D& Solution)
{
    const std::size_t NumRows = TridiagonalMatrix.SizeM();

    if (NumRows == 0)
    {
```

```
        throw std::invalid_argument(
            "ThomasAlgorithm::BackSubstitution: "
            "diagonal vectors must not be empty.");
    }

    if (TridiagonalMatrix.SizeU() != NumRows
        || RHS_Obj.GetSize() != NumRows
        || Solution.Size() != NumRows)
    {
        throw std::invalid_argument(
            "ThomasAlgorithm::BackSubstitution: "
            "all vectors must have the same size.");
    }

    const double LastDiagonalValue = TridiagonalMatrix.GetMidValue(NumRows - 1);

    if (std::fabs(LastDiagonalValue) < 1.0e-14)
    {
        throw std::runtime_error(
            "ThomasAlgorithm::BackSubstitution: "
            "zero or near-zero pivot encountered.");
    }

    Solution.SetValue(NumRows - 1,RHS_Obj.GetValue(NumRows - 1) / LastDiagonalValue);

    for (std::ptrdiff_t RowIndex = static_cast<std::ptrdiff_t>(NumRows) - 2; RowIndex >=
    {
        const std::size_t i = static_cast<std::size_t>(RowIndex);

        const double DiagonalValue = TridiagonalMatrix.GetMidValue(i);

        if (std::fabs(DiagonalValue) < 1.0e-14)
        {
            throw std::runtime_error(
                "ThomasAlgorithm::BackSubstitution: "
                "zero or near-zero pivot encountered.");
        }

        const double NewSolution =(RHS_Obj.GetValue(i) - TridiagonalMatrix.GetUValue(i)*
            / DiagonalValue;

        Solution.SetValue(i, NewSolution);
    }
}
```